

Software Options for Bayesian Modelling

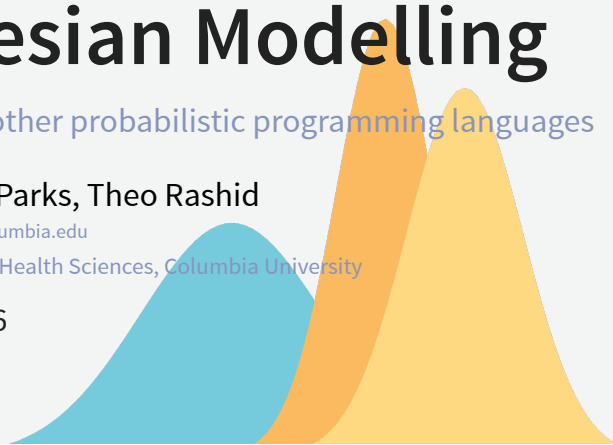
INLA and other probabilistic programming languages

Robbie M. Parks, Theo Rashid

robbie.parks@columbia.edu

Environmental Health Sciences, Columbia University

Aug 6, 2026



They wrote their posterior probability density

$$\begin{aligned}
 y_i &\sim \text{Normal}(\mu_i, \sigma) \\
 \mu_i &= \alpha + \mathbf{x}_i^T \beta \\
 \alpha &\sim \text{Normal}(0, 10) \\
 \beta &\sim \text{Normal}(0, 1) \\
 \sigma &\sim \text{Uniform}(0, 100)
 \end{aligned}$$

What did people do before packages?

They wrote their posterior probability density

```

1 import numpy as np
2 from scipy.stats import norm, uniform
3 from scipy.special import logsumexp
4
5 def logpdf(params, data):
6     log_prior = (
7         norm.logpdf(params.alpha, loc=0.0, scale=10.0) +
8         uniform.logpdf(params.sigma, low=0.0, high=100.0) +
9         np.sum(norm.logpdf(params.beta, loc=0.0, scale=1.0))
10    )
11
12    mu = params.alpha + jnp.matmul(x, params.beta)
13    log_likelihood = norm.logpdf(data, mu, params.sigma)
14
15    return logsumexp(log_prior + log_likelihood, axis=0)

```

They wrote their own samplers

```

1 import numpy as np
2
3 def rw_metropolis_kernel(logpdf, data, position, log_prob):
4     move_proposals = np.random.normal(0, 0.1, size=position.shape)
5     proposal = position + move_proposals
6     proposal_log_prob = logpdf(proposal, data)
7
8     log_uniform = np.log(np.random.uniform(low=0, high=1))
9     do_accept = log_uniform < proposal_log_prob - log_prob
10
11     position = np.where(do_accept, proposal, position)
12     log_prob = np.where(do_accept, proposal_log_prob, log_prob)
13     return position, log_prob

```

Probabilistic programming languages

We (probably) do not need to do this

- We will all (probably) make a mistake.
- Why write code that people have written before?
- Do you enjoy calibrating samplers?

WinBUGS

Write models in the BUGS language

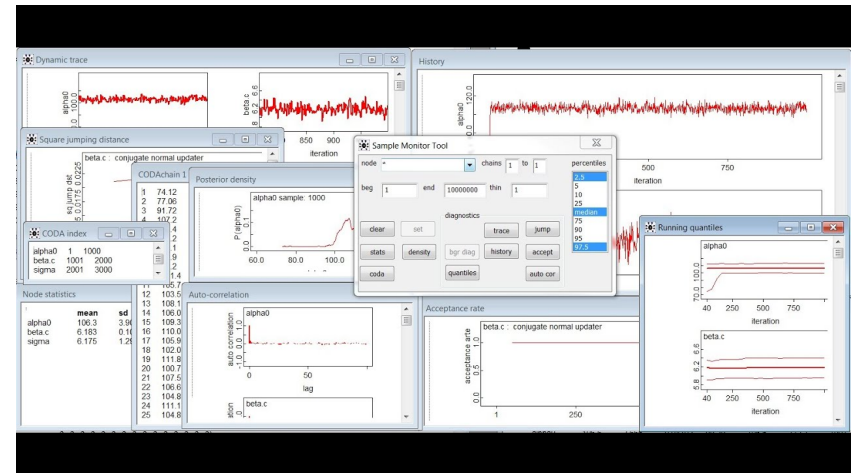
- BUGS stands for Bayesian inference Using Gibbs Sampling.
- Syntax may look familiar...

```

1 model(
2   # prior
3   alpha ~ dnorm(0, pow(10, -2))
4   sigma ~ dunif(0, 100)
5   tau <- pow(sigma, -2) # precision
6   for (k in 1:p) beta[k] ~ dnorm(0, 1)
7
8   # likelihood
9   for (i in 1:n) {
10    y[i] ~ dnorm(mu[i], tau)
11    mu[i] <- alpha + inprod(beta[1:p], x[i, 1:p])
12  }
13 )

```

Output looks like this...



What do we think of WinBugs nowadays?

- Point and click
- See your chains as they run
- Slow
- Old school
- Not really used widely these days...

Stan

Model in Stan (inspired by C++)

```
data {
  int<lower=0> N;    // number of data items
  int<lower=0> P;    // number of predictors
  matrix[N, K] x;   // predictor matrix
  vector[N] y;      // outcome vector
}
parameters {
  real alpha;        // intercept
  vector[P] beta;    // coefficients for predictors
  real<lower=0> sigma; // error scale
}
transformed parameters {
  vector[N] mu;
  mu = alpha + X * beta;
}
```

Stan, not STAN

NIMBLE

The grandfather of PPLs

- The statistician's choice
- Reliable, battle-tested samplers (including NUTS)
- Interface with python, R or Julia
- Using `brms` or `rstanarm`, you can write models like base R (like `y ~ 1 + x`) and the inference is done in Stan.
- Difficult to extend
- Writing things like `int<lower=0> N`

NIMBLE model is like WinBUGS, but more flexible.

- You are now quite familiar with NIMBLE syntax.

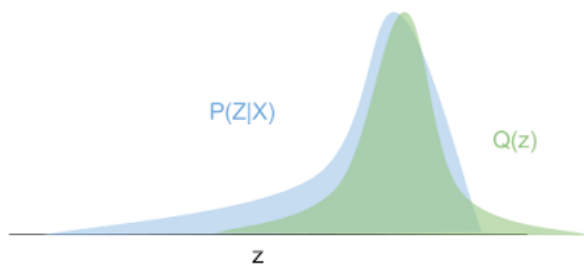
```
1 model(
2   # prior
3   alpha ~ dnorm(0, sd = 10)
4   sigma ~ dunif(0, 100)
5   beta[1:k] ~ dnorm(0, 1)
6
7   # likelihood
8   for (i in 1:n) {
9     y[i] ~ dnorm(mu[i], sd = sigma)
10    mu[i] <- alpha + (beta[1:p] %*% x[i, 1:p])[1,1]
11  }
12 )
```

We like NIMBLE!

- Compiles R to C++ for speed and scalability.
- Automatically finds conjugate relationships.
- Can pick a different sampler for each parameter.
- Good for teaching the fundamentals of Bayesian inference.
- If the sampler is not conjugate, it can mix poorly.

Variational inference

- Turn the inference problem into an optimisation problem.
- Minimise the difference between true posterior distribution, $p(z|x)$, and variational distribution, $q(z)$.



Is sampling the only way (introducing INLA)?

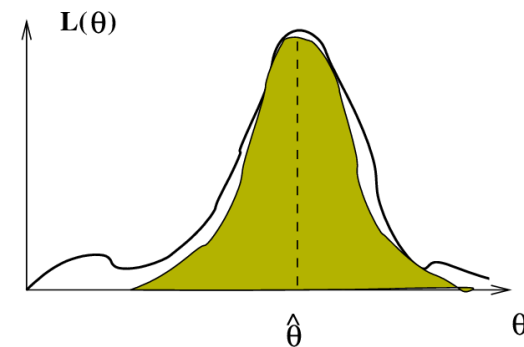
I've heard people talk about INLA. What is INLA?

- INLA stands for Integrated Nested Laplace Approximation.
- Approximate Bayesian inference for latent Gaussian models.
- Really good for spatial statistics.
- Really quick (if you don't have too many hyperparameters).
- Most real world cases are latent Gaussian – it just works.
- A growing community of users, 2 of whom are here today.

How does INLA work?

INLA computes accurate and fast approximations to the posterior marginals of the components of the latent Gaussian variables.

How does INLA work?



- The Laplace Approximation (**IN'LA'**) means that we approximate the latent field with a Gaussian. Most things in the real world are Gaussian, so this works well.

Do I fit models like all the PPLs above?

- No, actually, the syntax is quite similar to base R.
- Use R package **R-INLA** to fit models.
- For example, this is how to specify a model with an intercept and a random effect:

```
1 formula_iid <- y ~ 1 + f(x, model = "iid")
```

- So it's easier to specify models, and there's a lot of built in functionality.
- **But** it's more difficult to *customise*.

Running INLA

- We will focus on **INLA** as it is very useful and practical, and we will use it for the remainder of the course.
- First, write the model:

```
1 formula_iid <- y ~ 1 + f(x, model = "iid")
```

Running INLA

- Then, run the model using the `inla` function in R:

```
1 model <- inla(
2   formula_iid,
3   data = data,
4   E = expected,
5   family = "poisson",
6   control.predictor = list(link = 1), # log-link
7   control.compute = list(config = TRUE) # so that we can simulate
8                                     # draws of posterior
9 )
```

Previous examples from NIMBLE in INLA

Customising priors in INLA

- If you don't specify priors in INLA, it will choose some default ones.
- The formula from before gives us an IID effect, but no control over the hyperprior for the variance of the effect.

```
1 formula_iid <- y ~ 1 + f(x, model = "iid")
```

- INLA usually chooses some good defaults for you, but here, we can put a non-default prior on the precision ($1/\sigma^2$).

```
1 formula_iid_priors <- deaths ~ 1 + f(SIGLA, model = "iid",
2   hyper = list(prec = list(prior = "loggamma", param = c(0.01, 0.01))))
```

Linear regression

- NIMBLE formula:

```
1 code <- nimbleCode({
2   # priors for parameters
3   alpha ~ dnorm(0, sd = 100) # prior for alpha
4   beta1 ~ dnorm(0, sd = 100) # prior for beta1
5   beta2 ~ dnorm(0, sd = 100) # prior for beta2
6   beta3 ~ dnorm(0, sd = 100) # prior for beta3
7   sigma ~ dunif(0, 100) # prior for variance components
8
9   # regression formula
10  for (i in 1:n) {
11    # n is the number of observations we have in the data
12    mu[i] <- alpha + beta1 * x1[i] + beta2 * x2[i] + beta3 * x3[i] # manual
13    y[i] ~ dnorm(mu[i], sd = sigma)
14  }
15 })
```

Linear regression

- INLA formula (default priors):

```
1 formula_linear <- y ~ 1 + x1 + x2 + x3
```

- Run model using inla function:

```
1 model_linear <- inla(
2   formula_linear,
3   data = data,
4   family = "gaussian")
```

Linear regression

- INLA formula (custom priors):

```
1 formula_linear <- y ~ 1 + x1 + x2 + x3
```

Linear regression

- Run model using inla function:

```
1 model_linear <- inla(
2   formula_linear,
3   data = data,
4   family = "gaussian",
5   control.fixed = list(
6     mean = 0,
7     prec = 0.0001
8   ),
9   control.family = list(
10    hyper = list(
11      prec = list(
12        prior = "loggamma",
13        param = c(1, 0.0001)
14      )
15    )
16  )
17 )
```

Hierarchical model

- NIMBLE formula:

```
1 partial_pooling_model <- nimbleCode({
2   # priors
3   alpha ~ dnorm(0, 5)
4   sigma_p ~ T(dnorm(0, 1), 0, Inf) # half-normal
5
6   for (j in 1:Np) {
7     theta[j] ~ dnorm(0, sd = sigma_p)
8   }
9
10  # likelihood
11  for (i in 1:N) {
12    y[i] ~ dpois(mu[i])
13    log(mu[i]) <- log(E[i]) + alpha + theta[province[i]]
14  }
15 })
```

Hierarchical model

- INLA formula (custom priors):

```
1 formula_partial_pooling <- y ~ 1 + f(SIGLA, model = "iid",
2   hyper = list(
3     prec = list(
4       prior = "pc.prec",
5       param = c(1, 0.01)
6     )
7   )
8 )
```

Hierarchical model

- Run model using inla function:

```
1 model_partial_pooling <- inla(
2   formula_partial_pooling,
3   data = data,
4   E = expected,
5   family = "poisson",
6   control.predictor = list(link = 1),
7   control.compute = list(config = TRUE)
8 )
```

Non-linear exposure-response

- NIMBLE formula:

```
1 Model3_ecoCode <- nimbleCode({
2   # priors
3   alpha ~ dflat() # vague prior (Unif(-inf, +inf))
4   overallRR <- exp(alpha) # overall RR across study region
5
6   for (j in 1:K) {
7     beta[j] ~ dnorm(0, tau = 1)
8   }
9
10  # likelihood
11  for (i in 1:N) {
12    O[i] ~ dpois(mu[i]) # Poisson likelihood for observed counts
13    log(mu[i]) <- log(E[i]) + alpha + inprod(beta[], X[i, ])
14  }
15 })
```

Non-linear exposure-response

- INLA formula:

```
1 formula_non_linear <- 0 ~ 1 + X1 + X2 + X3
```

Non-linear exposure-response

- Run model using inla function:

```
1 model_non_linear <- inla(
2   formula = formula_non_linear,
3   family = "poisson",
4   data = covid_data,
5   offset = E,
6   control.fixed = list(
7     mean = c("(Intercept)" = 0, X1 = 0, X2 = 0, X3 = 0),
8     prec = c("(Intercept)" = 1e-6, X1 = 1, X2 = 1, X3 = 1)
9   ),
10  control.compute = list(dic = TRUE, waic = TRUE, cpo = TRUE)
11 )
```

I think I'm INLA (sorry)

- Good, we'll cover more in the practical.

Spatial model.

- You will see in lab!

Getting ready for the lab

The lab for this session

- The goal of this lab is to use [INLA](#) to run some hierarchical models.
- During this lab session, we will:
 1. Translate a model from [NIMBLE](#) into [INLA](#);
 2. Fit spatial models with [INLA](#);
 3. Learn how to work with [INLA](#) objects; and
 4. See how to write models in different probabilistic programming languages.

Questions?